

Network Sniffer — How It Works

A short technical report explaining the Scapy-based packet capture script

1. Purpose

This script is a lightweight packet sniffer built with the Scapy library. It captures live traffic on a network interface, inspects each packet's headers, and prints a readable summary showing the source/destination IP addresses, the transport-layer protocol, and (where present) the raw payload. It is intended as a learning tool for understanding how data flows through the network stack — IP layer down to TCP/UDP/ICMP — rather than as a production monitoring tool.

2. Requirements

- Python 3 with the **scapy** package installed (`pip install scapy`).
- Root/administrator privileges — raw socket capture needs elevated rights (`sudo python3 sniffer.py` on Linux/macOS; on Windows, **Npcap** must be installed and the script run as Administrator).
- A live network interface with traffic on it (Wi-Fi/Ethernet adapter).

3. Code Walkthrough

3.1 Imports and setup

The script imports `sniff`, `IP`, `TCP`, `UDP`, `ICMP`, `Raw` from Scapy. `sniff()` is the capture engine; the layer classes (`IP`, `TCP`, `UDP`, `ICMP`, `Raw`) are used to check which protocol layers are present in each captured packet and to pull out their fields. Scapy's own runtime logger is set to `ERROR` level so IPv6-related warnings do not clutter the output.

3.2 `packet_callback(packet)` — the per-packet handler

This function runs once for every packet Scapy captures. It is passed into `sniff()` as the `prn` (process function) argument.

- **Filter non-IP packets:** if the packet has no IP layer, the function returns immediately — ARP frames and other non-IP traffic are skipped.
- **Read IP header fields:** `ip.src` and `ip.dst` give the source and destination addresses; `ip.proto` is the protocol number, mapped to a readable name (1 = ICMP, 6 = TCP, 17 = UDP) via a small dictionary.
- **Inspect the transport layer:** the function checks, in order, whether a TCP, UDP, or ICMP layer is present and extracts the relevant fields — source/destination port and TCP flags for TCP, ports for UDP, type/code for ICMP.
- **Extract payload:** if a `Raw` layer exists (the packet carries application data), the first 100 bytes are decoded as UTF-8 text where possible; if decoding fails (binary data), it falls back to a hex string.
- **Print a summary block** for the packet, and any exception during processing is caught so one malformed packet cannot crash the whole capture loop.

3.3 `main()` — starting the capture

`main()` calls `sniff(prn=packet_callback, store=False, filter="ip")`. Key arguments: `prn` is the callback run per packet; `store=False` stops Scapy from keeping every packet in memory (important for long-running captures); `filter="ip"` is a BPF (Berkeley Packet Filter) expression applied at the kernel level so only IPv4 traffic reaches Python at all. `Ctrl+C` is caught to stop the sniffer cleanly.

4. Running the Script

On Linux/macOS: `sudo python3 sniffer.py`. On Windows: install Npcap, then run the terminal as Administrator before executing the script. Traffic will start printing immediately; generate some activity (browse a site, ping a host) to see packets flow. Press Ctrl+C to stop.

5. Sample Output

```
=====
Source IP      : 192.168.1.10
Destination IP : 142.250.183.14
Protocol       : TCP
Transport      : TCP | Src Port: 51322 | Dst Port: 443 | Flags: S
=====
```

6. Protocol Fields Captured

Layer	Fields Extracted	Purpose
IP	src, dst, proto	Identify sender/receiver and next-layer protocol
TCP	sport, dport, flags	Identify connection endpoints and connection state (SYN, ACK, etc.)
UDP	sport, dport	Identify connectionless service endpoints (DNS, DHCP, etc.)
ICMP	type, code	Identify control messages (e.g., echo request/reply for ping)
Raw	load (payload bytes)	Show application-layer data carried inside the packet

7. Notes and Limitations

- Only IPv4 traffic is captured (the BPF filter is "ip"); IPv6 packets are excluded.
- Payloads on most modern web/app traffic are encrypted (TLS), so decoded text will often look like binary garbage — this is expected, not a bug.
- This tool only reads traffic already reaching the local interface; it does not perform ARP spoofing or any active interception.
- Capturing network traffic you do not own or have permission to monitor may be illegal — use only on networks and devices you control or are authorized to test.